Detailed Study Notes

Preserving State in Web Applications

ASP.NET State Management & List-Bound Controls

1. Preserving State in Web Applications

1.1 What is State?

In the context of web applications, 'state' refers to any information that needs to be remembered and persisted across multiple HTTP requests. Examples include a user's login status, items in a shopping cart, form input values, user preferences, and navigation history.

1.2 The Stateless Nature of HTTP

HTTP (HyperText Transfer Protocol) is inherently stateless. This means that each request sent from a browser to a web server is completely independent — the server has no built-in memory of previous requests from the same client. Once a server sends a response, it forgets about that interaction entirely.

This stateless behavior creates a fundamental challenge: how can a web application remember who a user is across multiple page requests?

Why HTTP is Stateless:

- Designed for simplicity and scalability — servers don't need to maintain connection state.
- Each HTTP request contains all the information needed to fulfill it independently.
- This makes HTTP servers easy to scale horizontally (add more servers).
- However, it makes session-aware applications harder to build.

1.3 Page-Level State (ViewState)

ASP.NET introduced ViewState as a mechanism to persist page-level state between postbacks. When a form is submitted (postback), ASP.NET serializes the values of all controls on the page and stores them in a hidden field called __VIEWSTATE.

The __VIEWSTATE field is a Base64-encoded string embedded as a hidden input in the HTML form. On every postback, this value is sent back to the server, and ASP.NET uses it to restore control values.

How ViewState Works:

```
<!-- ViewState is automatically added to ASP.NET pages -->
<input type='hidden' name='__VIEWSTATE' id='__VIEWSTATE'
  value='/wEPDwUKLTEyMTY5MDY2Mg9kFgICAw8...' />
```

EnableViewState	Set to false on controls/pages to disable (reduces page size).
ViewStateMode	Enabled or Disabled — inheritable control-level property.
__VIEWSTATE	Hidden HTML field holding the serialized state.

Advantages:

- No server-side storage required — all data travels with the page.
- Automatically maintained by the ASP.NET runtime.
- Works per-page without any extra configuration.

Disadvantages:

- Increases page size (can become very large for complex pages).
- Data is sent over the network on every postback.
- Not suitable for sensitive data (only encoded, not encrypted by default).
- Does not persist across different pages — only within a single page.

2. Using Cookies to Preserve State

2.1 What are Cookies?

Cookies are small pieces of text data that a web server sends to the client's browser. The browser stores these cookies and automatically sends them back to the server with every subsequent request to the same domain. Cookies are the oldest and most widely used mechanism for maintaining state in web applications.

2.2 Types of Cookies

Persistent Cookie	Stored on the user's disk with an expiry date. Survives browser restarts until expiry.
Session Cookie	Stored in browser memory only. Deleted when the browser is closed (no Expires attribute).
Secure Cookie	Only transmitted over HTTPS connections (Secure flag set).
HttpOnly Cookie	Cannot be accessed by JavaScript — protects against XSS attacks.

2.3 Working with Cookies in ASP.NET

Setting a Cookie:

```
// Single-value cookie
HttpCookie cookie = new HttpCookie("username");
cookie.Value = "Alice";
cookie.Expires = DateTime.Now.AddDays(30);
cookie.HttpOnly = true;  // Protect against XSS
cookie.Secure = true;    // HTTPS only
Response.Cookies.Add(cookie);

// Multi-value cookie (sub-keys)
HttpCookie profile = new HttpCookie("profile");
profile["name"] = "Alice";
profile["role"] = "admin";
profile.Expires = DateTime.Now.AddDays(7);
Response.Cookies.Add(profile);
```

Reading a Cookie:

```
// Check if cookie exists before reading
if (Request.Cookies["username"] != null)
{
    string user = Request.Cookies["username"].Value;
}

// Reading sub-key
string role = Request.Cookies["profile"]["role"];
```

Modifying a Cookie:

```
// To update, re-create and add (overwrites existing)
HttpCookie cookie = Request.Cookies["username"];
cookie.Value = "Bob";
cookie.Expires = DateTime.Now.AddDays(30);
Response.Cookies.Add(cookie);
```

Deleting a Cookie:

```
// Set expiry to past date to delete
HttpCookie cookie = new HttpCookie("username");
cookie.Expires = DateTime.Now.AddDays(-1);
Response.Cookies.Add(cookie);
```

2.4 Limitations of Cookies

- Maximum size is approximately 4 KB per cookie.
- Most browsers limit the number of cookies per domain to around 50.
- Users can disable cookies in their browser settings.
- Cookies are sent with every request, increasing HTTP overhead.
- Not suitable for storing sensitive information without encryption.
- Third-party cookie restrictions in modern browsers (privacy regulations).

3. ASP.NET Session State

3.1 What is Session State?

Session State is a server-side mechanism for storing per-user data across multiple requests. Unlike cookies (which store data on the client), session data is stored on the server. Only a unique session identifier (Session ID) is stored on the client side, typically as a cookie.

Each user who visits a web application gets a unique Session ID — a 120-bit random number encoded as a 20-character string. This ID is used to look up the user's session data on the server.

3.2 How Session State Works (Flow)

1. User makes the first request to the web application.
2. Server creates a new Session object and assigns a unique Session ID (120-bit random value).
3. Session data is stored server-side (in memory, state server, or database).
4. The Session ID is sent to the client as a cookie named ASP.NET_SessionId.
5. On every subsequent request, the browser sends the Session ID cookie.
6. Server reads the Session ID and retrieves the matching session data.

3.3 Using Session State in Code

Storing Values:

```
Session["username"] = "Alice";
Session["userId"] = 101;
Session["role"] = "Admin";
```

Retrieving Values:

```
// Always cast when retrieving — Session returns object
string user = (string)Session["username"];
int id = (int)Session["userId"];

// Safe retrieval with null check
if (Session["username"] != null)
{
    string user = Session["username"].ToString();
}
```

Removing Values:

```
Session.Remove("username");  // Remove a specific key
Session.Clear();              // Remove all items
Session.Abandon();            // End session immediately
```

3.4 Session State Properties

SessionID	Unique 120-bit identifier for the current session (read-only).
Timeout	Minutes of inactivity before session expires. Default is 20 minutes.
Count	Number of items stored in the current session.
IsNewSession	True if the session was created with the current request.
IsReadOnly	True if the session is read-only.
Abandon()	Ends the session immediately and fires Session_End event.

`Clear()`	Removes all items but keeps the session alive.

3.5 Session Lifecycle Events (Global.asax)

```
void Session_Start(object sender, EventArgs e) {
    // Fires when a new session is created
    Application["activeSessions"] = (int)Application["activeSessions"] + 1;
}

void Session_End(object sender, EventArgs e) {
    // Fires when session ends (only for InProc mode)
    Application["activeSessions"] = (int)Application["activeSessions"] - 1;
}
```

4. Storing Objects in Session State

4.1 Storing Complex Types

Session State can store not just primitive values (strings, integers) but also complex objects such as custom classes, collections, and DataTables. This is especially useful for maintaining application-specific data structures like shopping carts, user profiles, or form wizard data.

4.2 The [Serializable] Attribute

When using InProc session mode, objects are stored as-is in memory. However, when using StateServer or SQLServer session modes, the objects must be serialized (converted to a byte stream) before being transferred or stored. This requires the class to be marked with the [Serializable] attribute.

```
[Serializable]
public class ShoppingCart
{
    public List<CartItem> Items { get; set; }
    public decimal TotalAmount { get; set; }

    public ShoppingCart()
    {
        Items = new List<CartItem>();
    }

    public void AddItem(string name, decimal price, int qty)
    {
        Items.Add(new CartItem { Name = name, Price = price, Quantity = qty });
        TotalAmount += price * qty;
    }
}

[Serializable]
public class CartItem
{
    public string Name { get; set; }
    public decimal Price { get; set; }
    public int Quantity { get; set; }
}
```

4.3 Storing and Retrieving the Object

```
// Storing the cart
ShoppingCart cart = new ShoppingCart();
cart.AddItem("Laptop", 55000, 1);
cart.AddItem("Mouse", 850, 2);
Session["Cart"] = cart;

// Retrieving the cart
ShoppingCart myCart = (ShoppingCart)Session["Cart"];
if (myCart != null)
{
    // Safe to use
    lblTotal.Text = myCart.TotalAmount.ToString("C");
}
```

4.4 Best Practices

- Always mark classes as [Serializable] even if you currently use InProc mode — you might switch later.

- Avoid storing very large objects in session — they consume server memory for every active user.
- Consider storing only the key (e.g., user ID) in session and loading data fresh from the database.
- Call Session.Abandon() on logout to free memory immediately.
- Use Session_End in Global.asax to clean up resources when sessions expire.

5. Configuring Session State

5.1 The <sessionState> Element in web.config

Session State behavior is configured through the <sessionState> element in the web.config file. This allows you to choose where session data is stored, how long sessions last, and whether to use cookies or URL-based session IDs.

```
<configuration>
  <system.web>
    <sessionState
      mode="InProc"
      cookieless="false"
      timeout="20"
      stateConnectionString="tcpip=127.0.0.1:42424"
      sqlConnectionString="data source=localhost;Integrated Security=SSPI"
    />
  </system.web>
</configuration>
```

5.2 Session State Modes — Comparison

Mode	Storage Location	Persists App Restart?	Performance	Best Use Case
InProc	Web server memory	No	Fastest	Single server / development
StateServer	ASP.NET State Service	No	Moderate	Web farm (multiple servers)
SQLServer	SQL Server database	Yes	Slower	Mission-critical / durable
Custom	Custom provider	Depends	Varies	Special storage requirements
Off	Disabled	—	N/A	When session not needed

5.3 Configuration Attributes Explained

`mode`	Sets the storage provider: InProc, StateServer, SQLServer, Custom, or Off.
`cookieless`	Controls how Session ID is sent: false (cookie), true (URL), AutoDetect, UseDeviceProfile.
`timeout`	Number of idle minutes before the session expires. Default: 20.
`stateConnectionString`	Required for StateServer mode. Format: tcpip=host:port.
`sqlConnectionString`	Required for SQLServer mode. Standard connection string.
`cookieName`	Name of the session cookie. Default: ASP.NET_SessionId.
`regenerateExpiredSessionId`	Regenerates session ID when expired session ID received (security).

6. Out-of-Process State Server

6.1 What is Out-of-Process State?

Out-of-process session state refers to storing session data outside the web server's own process (w3wp.exe). This is necessary in web farm scenarios where multiple web servers handle requests from the same user. Since InProc stores data in the web server's memory, different servers would not share session data.

ASP.NET provides two out-of-process options: StateServer (using the ASP.NET State Service Windows service) and SQLServer (using a SQL Server database).

6.2 Setting Up ASP.NET State Service (StateServer Mode)

Step 1 — Start the ASP.NET State Service:

```
REM Command Prompt (run as Administrator)
net start aspnet_state

REM Or via Services.msc — find 'ASP.NET State Service'
REM Set Startup Type to 'Automatic' for persistence
```

Step 2 — Configure web.config:

```
<sessionState
  mode="StateServer"
  stateConnectionString="tcpip=127.0.0.1:42424"
  timeout="30"
/>

<!-- For a remote state server, use the server's IP -->
<sessionState
  mode="StateServer"
  stateConnectionString="tcpip=192.168.1.100:42424"
/>
```

Step 3 — Firewall Configuration:

- The default port for ASP.NET State Service is 42424.
- This port must be open on both the web server and the state server.
- In web farm setups, all web servers point to the same state server IP/port.

Step 4 — Serializable Objects:

- All objects stored in session MUST be marked [Serializable] because they are marshaled out of the web server process via TCP/IP.
- This is the most common mistake when migrating from InProc to StateServer.

6.3 Advantages & Disadvantages

Advantage	Session data survives IIS/application pool restarts.
Advantage	Supports web farm deployments (multiple web servers).
Advantage	State service is lightweight and fast.
Disadvantage	Session data is lost if the State Service itself is restarted.
Disadvantage	Objects must be serializable — extra coding requirement.

Disadvantage	Higher overhead than InProc due to process boundary crossing.

7. Storing Session State in SQL Server

7.1 Why Use SQL Server for Session State?

SQL Server session state provides the most durable form of state management. Session data is persisted in a SQL Server database, meaning it survives not just IIS restarts but also complete server reboots. This is critical for high-availability applications.

7.2 Setup Procedure

Step 1 — Create the Session Database:

ASP.NET ships with a command-line tool called aspnet_regsql.exe that creates the required database and tables in SQL Server.

```
REM Run from Visual Studio Command Prompt
aspnet_regsql.exe -S localhost -E -ssadd -sstype p

REM Flags:
REM  -S    : SQL Server name
REM  -E    : Use Windows authentication
REM  -ssadd : Add session state support
REM  -sstype p : Use persisted (permanent) database
REM             (use 't' for temporary - recreated each restart)
```

This command creates a database named ASPState (for persistent type) with two tables: ASPStateTempApplications and ASPStateTempSessions.

Step 2 — Configure web.config:

```
<sessionState
  mode="SQLServer"
  sqlConnectionString="data source=localhost;
    Integrated Security=SSPI"
  timeout="30"
/>

<!-- Or using SQL authentication: -->
<sessionState
  mode="SQLServer"
  sqlConnectionString="data source=localhost;
    User Id=sessionUser;Password=secret123"
/>
```

Step 3 — Serializable Classes:

- Same requirement as StateServer — all session objects must be [Serializable].
- SQL Server uses binary serialization to store objects in the database.

7.3 Key Characteristics

Database Created	ASPState (by default) — contains session tables.
Persists Restart?	Yes — data survives IIS, app pool, and OS restarts.
Web Farm Support	Yes — multiple web servers share one SQL Server.
Cleanup	ASP.NET automatically deletes expired sessions via SQL Agent jobs.
Performance	Slower than InProc/StateServer due to database I/O.

8. Using Cookieless Session IDs

8.1 The Problem

Some users disable cookies in their browsers, or certain environments (like some mobile browsers or kiosks) may not support cookies. In these cases, the standard Session ID cookie cannot be set, and the web application loses the ability to track sessions.

8.2 The Solution — URL-Embedded Session IDs

When cookieless mode is enabled, ASP.NET embeds the Session ID directly in the URL path instead of using a cookie. The Session ID appears as an extra path segment between the hostname and the page name.

Example URL with Cookieless Session ID:

```
// Normal URL:
http://www.example.com/store/checkout.aspx

// Cookieless URL (Session ID embedded):
http://www.example.com/(lit3py55t21z5v55vlm25s55)/store/checkout.aspx

// The 24-character value in parentheses is the Session ID
```

8.3 Configuration Options

```
<!-- Always use URL-based Session IDs -->
<sessionState cookieless="true" />

<!-- Always use cookies (default) -->
<sessionState cookieless="false" />

<!-- Auto-detect: uses cookie if available, URL otherwise -->
<sessionState cookieless="AutoDetect" />

<!-- Based on browser/device capabilities -->
<sessionState cookieless="UseDeviceProfile" />
```

`true (UseUri)`	Always embed Session ID in URL — never use cookies.
`false (UseCookies)`	Always use cookies for Session ID (default behavior).
`AutoDetect`	Server checks if browser accepts cookies; uses URL if not.
`UseDeviceProfile`	Uses ASP.NET browser capabilities database to decide.

8.4 ASP.NET Automatic URL Rewriting

When cookieless mode is enabled, ASP.NET automatically handles URL rewriting. All links generated by server controls (HyperLink, Response.Redirect, etc.) will automatically include the Session ID in the URL. Developers do not need to manually modify links.

```
// Response.Redirect works automatically:
Response.Redirect("~/products.aspx");
// Becomes: /(<sessionId>)/products.aspx
```

8.5 Security Considerations

- Session ID is visible in the URL — can be seen in browser history, server logs, and Referer headers.
- If a user copies and shares the URL, they share their session — session hijacking risk.
- Always use HTTPS when using cookieless sessions to prevent ID interception.
- Regenerate Session IDs after login: Response.Redirect(Request.RawUrl).
- Consider cookieless sessions only when cookies are truly unavailable.

9. Application State

9.1 What is Application State?

Application State is a global, shared memory store that is accessible by all users and all sessions of a web application. Unlike Session State (which is per-user), Application State has a single instance shared across the entire application. It lives in the HttpApplicationState object, accessible via the Application property.

Application State is best used for storing data that: (a) does not change often, (b) is the same for all users, and (c) is computationally expensive to retrieve from a database on every request.

9.2 Using Application State

```
// Storing application-level data
Application["appName"] = "MyWebApp";
Application["maxUsers"] = 100;

// Retrieving values
string name = (string)Application["appName"];
int max = (int)Application["maxUsers"];

// Removing a value
Application.Remove("maxUsers");
```

9.3 Thread Safety — Lock and UnLock

Application State is shared across all concurrent requests. Without protection, two simultaneous requests could read and write the same value, causing race conditions (data corruption). ASP.NET provides Lock() and UnLock() methods to prevent this.

```
// ALWAYS Lock before writing to Application State
Application.Lock();
try
{
    int hits = (int)Application["hitCount"];
    Application["hitCount"] = hits + 1;
}
finally
{
    Application.UnLock();  // Always unlock, even on exception
}

// Reading does NOT require locking (atomic in .NET)
int currentHits = (int)Application["hitCount"];
```

9.4 Initialization in Global.asax

```
void Application_Start(object sender, EventArgs e)
{
    // Initialize application-level values
    Application["hitCount"] = 0;
    Application["startTime"] = DateTime.Now;
    Application["siteName"] = "My ASP.NET App";
}
```

9.5 Session State vs. Application State — Comparison

Feature	Session State	Application State

Scope	Per-user (one per session)	Global (shared by all users)
Storage	Server memory / DB	Web server memory only
Lifetime	Until timeout or Abandon()	Application lifetime
Thread Safety	Per-user (inherently safe)	Requires Lock()/UnLock()
Web Farm Support	Yes (StateServer/SQL)	No — each server has own copy
Best For	User-specific data (cart, login)	Site-wide counters, config, cache

9.6 Limitations of Application State

- Not suitable for web farm deployments — each web server has its own Application State.
- All data is lost when the application restarts.
- No persistence — not stored to disk.
- Overuse can cause memory pressure, especially with large objects.
- For modern applications, consider using distributed caches (Redis, Memcached) instead.

10. Overview of List-Bound Controls

10.1 What are List-Bound Controls?

List-bound controls (also called data-bound controls) are ASP.NET server controls that can bind to a data source and automatically render a list or grid of items. They eliminate the need to manually write loops to build HTML tables or lists — the runtime handles the rendering based on templates or column definitions.

All list-bound controls share a common data-binding pattern: assign a DataSource and call DataBind().

```
// Common pattern for all list-bound controls:
control.DataSource = GetDataFromDatabase();
control.DataBind();
```

10.2 Common Data Sources

- DataTable / DataSet — result of database queries via ADO.NET.
- List<T> — strongly typed generic lists (e.g., List<Product>).
- Array — simple arrays of objects.
- IEnumerable — any enumerable collection.
- ObjectDataSource, SqlDataSource — declarative data source controls.

10.3 The Three Main List Controls — Overview

Control	Output Format	Templates	Paging/Sorting	Best Use
GridView	Auto HTML table	Optional column templates	Built-in	Tabular admin data
DataList	Table-based layout	Required item templates	Manual	Multi-column product grids
Repeater	Pure HTML (no wrapper)	Required all templates	Manual	Custom HTML output

10.4 Data Binding Expressions

Inside templates, you use data binding expressions to display field values:

`<%# Eval("FieldName") %>`	One-way binding — reads the value from the data source.
`<%# Bind("FieldName") %>`	Two-way binding — reads and writes (used with edit templates).
`<%# Container.ItemIndex %>`	Zero-based index of the current item in the list.
`<%# Container.DataItem %>`	The full data row/object for the current item.

11. Creating a Repeater Control

11.1 What is a Repeater?

The Repeater control is the most flexible and lightweight of ASP.NET's list-bound controls. It generates no HTML wrapper by itself — the output is exactly what you put in the templates. This gives you complete control over the HTML structure, making it ideal when you need to produce custom markup like unordered lists, definition lists, or card grids.

11.2 Repeater Templates

HeaderTemplate	Rendered once before any items. Use for opening tags like <table> or <ul>.
ItemTemplate	Rendered once for each item in the data source. Contains data binding expressions.
AlternatingItemTemplate	Optional — rendered for every other item (odd-numbered items).
SeparatorTemplate	Rendered between items (not before first or after last).
FooterTemplate	Rendered once after all items. Use for closing tags like </table> or </ul>.

11.3 Creating a Repeater — Complete Example

ASPX Markup:

```
<asp:Repeater ID="rptProducts" runat="server"
    OnItemCommand="rptProducts_ItemCommand">

  <HeaderTemplate>
    <ul class="product-list">
  </HeaderTemplate>

  <ItemTemplate>
    <li>
      <strong><%# Eval("ProductName") %></strong>
      — Price: <%# Eval("Price", "{0:C}") %>
      <asp:LinkButton CommandName="Select"
        CommandArgument='<%# Eval("ProductId") %>'
        runat="server">View Details</asp:LinkButton>
    </li>
  </ItemTemplate>

  <AlternatingItemTemplate>
    <li style="background-color:#f0f0f0;">
      <strong><%# Eval("ProductName") %></strong>
      — Price: <%# Eval("Price", "{0:C}") %>
    </li>
  </AlternatingItemTemplate>

  <SeparatorTemplate>
    <li><hr /></li>
  </SeparatorTemplate>

  <FooterTemplate>
    </ul>
  </FooterTemplate>
```

```
</asp:Repeater>
```

Code-Behind (C#):

```
protected void Page_Load(object sender, EventArgs e)
{
    if (!IsPostBack)
    {
        BindRepeater();
    }
}

private void BindRepeater()
{
    // Using a List<T> as data source
    List<Product> products = ProductDB.GetAll();
    rptProducts.DataSource = products;
    rptProducts.DataBind();
}

protected void rptProducts_ItemCommand(object source,
    RepeaterCommandEventArgs e)
{
    if (e.CommandName == "Select")
    {
        int productId = Convert.ToInt32(e.CommandArgument);
        Response.Redirect("ProductDetail.aspx?id=" + productId);
    }
}
```

11.4 Key Points about Repeater

- No auto-generated HTML — you control every byte of output.
- No built-in support for paging, sorting, or editing — must be implemented manually.
- Most lightweight of all list controls — best performance.
- ItemCommand event fires when a Button/LinkButton inside the template is clicked.
- Use Container.ItemIndex for zero-based item number; Container.DataItem for the raw row.

12. Creating a DataList Control

12.1 What is a DataList?

The DataList control renders data in a table-based layout with support for multiple columns. Unlike the Repeater, it generates an HTML <table> wrapper automatically. DataList gives you more layout control than GridView (which is strictly row/column) but less raw flexibility than Repeater.

DataList is ideal for product catalog pages, image galleries, or any layout where items should flow in a multi-column grid.

12.2 DataList Templates

HeaderTemplate	Rendered once before the data items.
ItemTemplate	Rendered for each data item (required).
AlternatingItemTemplate	Optional — rendered for alternating items.
EditItemTemplate	Rendered when an item's EditItemIndex matches the item index.
SelectedItemTemplate	Rendered for the item whose index matches SelectedIndex.
SeparatorTemplate	Rendered between items.
FooterTemplate	Rendered once after all data items.

12.3 Key Layout Properties

RepeatColumns	Number of columns to render. Default: 1.
RepeatDirection	Horizontal (fill rows first) or Vertical (fill columns first).
RepeatLayout	Table (renders a <table>) or Flow (renders tags).
EditItemIndex	Index of the item currently in edit mode. -1 means no item is editing.
SelectedIndex	Index of the currently selected item. -1 means none selected.
CellPadding	Pixels of padding inside each cell.
CellSpacing	Pixels of spacing between cells.

12.4 Complete DataList Example

ASPX Markup:

```
<asp:DataList ID="dlProducts" runat="server"
  RepeatColumns="3"
  RepeatDirection="Horizontal"
  RepeatLayout="Table"
  CellPadding="10"
  OnEditCommand="dlProducts_EditCommand"
  OnUpdateCommand="dlProducts_UpdateCommand"
  OnCancelCommand="dlProducts_CancelCommand">

  <HeaderTemplate>
    <h3>Product Catalog</h3>
```

```
  </HeaderTemplate>

  <ItemTemplate>
    <div class="product-card">
      <h4><%# Eval("Name") %></h4>
      <p>Price: <%# Eval("Price", "{0:C}") %></p>
      <asp:LinkButton CommandName="Edit" runat="server">Edit</asp:LinkButton>
    </div>
  </ItemTemplate>

  <EditItemTemplate>
    <div class="product-card editing">
      Name: <asp:TextBox ID="txtName" runat="server"
        Text='<%# Bind("Name") %>' /><br/>
      Price: <asp:TextBox ID="txtPrice" runat="server"
        Text='<%# Bind("Price") %>' /><br/>
      <asp:LinkButton CommandName="Update" runat="server">Save</asp:LinkButton>
      <asp:LinkButton CommandName="Cancel" runat="server">Cancel</asp:LinkButton>
    </div>
  </EditItemTemplate>

  <AlternatingItemTemplate>
    <div class="product-card alt">
      <h4><%# Eval("Name") %></h4>
      <p>Price: <%# Eval("Price", "{0:C}") %></p>
    </div>
  </AlternatingItemTemplate>

</asp:DataList>
```

Code-Behind (C#):

```
protected void Page_Load(object sender, EventArgs e)
{
    if (!IsPostBack) BindList();
}

private void BindList()
{
    dlProducts.DataSource = ProductDB.GetAll();
    dlProducts.DataBind();
}

protected void dlProducts_EditCommand(object source,
    DataListCommandEventArgs e)
{
    dlProducts.EditItemIndex = e.Item.ItemIndex;
    BindList();
}

protected void dlProducts_UpdateCommand(object source,
    DataListCommandEventArgs e)
{
    TextBox txtName = (TextBox)e.Item.FindControl("txtName");
    TextBox txtPrice = (TextBox)e.Item.FindControl("txtPrice");
    // Update database...
    dlProducts.EditItemIndex = -1;  // Exit edit mode
    BindList();
}

protected void dlProducts_CancelCommand(object source,
    DataListCommandEventArgs e)
{
    dlProducts.EditItemIndex = -1;  // Cancel edit
    BindList();
```

```
}
```

12.5 RepeatDirection Explained

When RepeatColumns is set to 3 and you have 6 items:

- Horizontal direction: Row 1 = [Item1, Item2, Item3], Row 2 = [Item4, Item5, Item6]
- Vertical direction: Col 1 = [Item1, Item4], Col 2 = [Item2, Item5], Col 3 = [Item3, Item6]

12.6 Repeater vs. DataList — Key Differences

Feature	Repeater	DataList
HTML Wrapper	None — pure templates	Always renders a <table>
Multi-Column	Manual via CSS/HTML	Built-in (RepeatColumns)
Edit Support	Manual only	Built-in EditItemTemplate
Performance	Lightest	Slightly heavier
Flexibility	Maximum	High (less than Repeater)
Use For	Custom HTML lists	Grid/catalog layouts

Quick Reference Summary

State Management Techniques at a Glance

Technique	Storage Location	Scope	Lifetime	Best For
ViewState	Hidden HTML field	Single page	One request/postback	Control values, form data
Cookies	Client browser	Across pages	Configurable	User preferences, tracking
Session State	Server (varies)	Per user	Until timeout/abandon	Login, cart, user data
Application State	Server memory	All users	App lifetime	Counters, global config

Session State Modes at a Glance

Mode	Persists Restart?	Web Farm?	Serialize?	Performance
InProc	No	No	No	Fastest
StateServer	No (service restart)	Yes	Yes	Fast
SQLServer	Yes	Yes	Yes	Moderate
Custom	Depends	Depends	Depends	Varies

List-Bound Controls at a Glance

Control	HTML Output	Multi-Column	Edit Support	Use Case
GridView	Auto <table> + columns	Yes (columns)	Yes (built-in)	Admin tables, reports
DataList	<table> wrapper	Yes (RepeatColumns)	Yes (EditItemTemplate)	Product grids, galleries
Repeater	Pure custom HTML	Manual	Manual	Custom lists, menus